

Object Oriented Programming (Templates)

Abbas Khalid

April 21, 2025

Table of contents

1 Function Templates

2 Class Templates

Example 1

```
1 void swap_char(char&,char&);
2 void swap_int(int&,int&);
3 void swap_double(double&,double&);
4
5 int main() {
6     char a = 'A'; char b = 'Z';
7     cout<<"Characters before call: "<<a<<" and "<<b<<endl;
8     swap_char(a,b) ;
9     cout<<"Characters after call: "<<a<<" and "<<b<<endl;
10
11     double x = 1.5; double y = 3.2;
12     cout<<"Doubles before call: "<<x<<" and "<<y<<endl;
13     swap_double(x,y);
14     cout<<"Doubles after call: "<<x<<" and "<<y<<endl;
15
16     int i = 2; int j = 5;
17     cout<<"Integers before call: "<<i<<" and "<<j<<endl;
18     swap_int(i,j);
19     cout<<"Integers after call: "<<i<<" and "<<j<<endl;
20     return 0;
21 }
```

Characters before call:A and Z
Characters after call:Z and A
Doubles before call:1.5 and 3.2
Doubles after call:3.2 and 1.5
Integers before call:2 and 5
Integers after call:5 and 2

Example 1 (Cont.)

```
1 void swap_char(char& a, char& b){
2     char temp;
3     temp = b;
4     b = a;
5     a = temp;
6 }
7
8 void swap_double(double& a, double& b){
9     double temp;
10    temp = b;
11    b = a;
12    a = temp;
13 }
14
15 void swap_int(int& a, int& b) {
16     int temp;
17     temp = b;
18     b = a;
19     a = temp;
20 }
```

Example 2

```
1 void Swap(char&,char&);
2 void Swap(int&,int&);
3 void Swap(double&,double&);
4
5 int main() {
6     char a = 'A'; char b = 'Z';
7     cout<<"Characters before call: "<<a<<" and "<<b<<endl;
8     Swap(a,b) ;
9     cout<<"Characters after call: "<<a<<" and "<<b<<endl;
10
11     double x = 1.5; double y = 3.2;
12     cout<<"Doubles before call: "<<x<<" and "<<y<<endl;
13     Swap(x,y);
14     cout<<"Doubles after call: "<<x<<" and "<<y<<endl;
15
16     int i = 2; int j = 5;
17     cout<<"Integers before call: "<<i<<" and "<<j<<endl;
18     Swap(i,j);
19     cout<<"Integers after call: "<<i<<" and "<<j<<endl;
20     return 0;
21 }
```

Characters before call:A and Z
Characters after call:Z and A
Doubles before call:1.5 and 3.2
Doubles after call:3.2 and 1.5
Integers before call:2 and 5
Integers after call:5 and 2

Example 2 (Cont.)

```
1 void Swap(char& a, char& b){
2     char temp;
3     temp = b;
4     b = a;
5     a = temp;
6 }
7
8 void Swap(double& a, double& b){
9     double temp;
10    temp = b;
11    b = a;
12    a = temp;
13 }
14
15 void Swap(int& a, int& b) {
16     int temp;
17     temp = b;
18     b = a;
19     a = temp;
20 }
```

Function Templates

- Overloaded functions normally perform similar or identical operations on different types of data. If the operations are identical for each type, they can be expressed more compactly and conveniently using generic functions.
- In a generic function, a single procedure can be applied to a wide range of data. Many algorithms are logically the same no matter what type of data is being operated upon. For example, the Bubble Sort is the same whether it is applied to an array of integers or an array of floats.
- The type of data that the function will operate upon is passed to it as a parameter.
- Based on the argument types provided explicitly or inferred from calls to this function, the compiler generates separate source-code functions to handle each function call appropriately.
- In essence, when we create a generic function we are creating a function that can automatically overload itself.

Function Templates

- A function template is a blueprint or a recipe for a function. The compiler uses it to generate a new member of a family of functions. The new function is created when it is first required. A function that is generated from a function template is described as an instance or an instantiation of the template.
- A function template definition begins with keyword `template` followed by a list of parameters to the function template enclosed in angle brackets (< and >).
- Each template parameter that represents a type must be preceded by either of the keywords `class` or `typename`.

```
1  template <class type>
2  returnType functionName (parameterList) {
3  // body of function
4  }
```

```
1  template <typename type>
2  returnType functionName (parameterList) {
3  // body of function
4  }
```


Function Templates

- The parameter type is used to specify the types of the function's parameters, to specify the return type of the function and to declare variables within the function. It does not necessarily have to be of `class` or `typename` type. It can be any of the fundamental types such as `int` or `long`, or it could be a user-defined type.
- The compiler creates a new version of the function by replacing type throughout the template definition with the particular type in order to create a particular instance from the template.
- The general form to call a function template is:

```
1 functionName<type> (argumentList) ;
```

- In particular, there is no need to specify a value for the template argument type because the compiler deduces the value of type from the arguments to the function call. In certain cases, the compiler is unable to deduce the template arguments (due to ambiguity) and hence is unable to choose which version of the function to use. In such cases, we must specify the template parameter explicitly.

Function Templates

- If we specify the parameter for the template explicitly when we call the function, the compiler no longer tries to deduce the type to replace type; it simply accepts what we specify. This allows us to control which version of the function is used.
- Note that a function template does not do anything by itself. It is a recipe or blueprint that the compiler uses to create a function definition from a function call.
- This process happens during compilation. The compiler uses a template to generate source code for a function definition, which it then compiles.
- The compiler only generates this instantiation of the template once. If a subsequent function call requires the same instance, then it calls the instance already created.
- In the following call, although we are passing an `int` type argument, the compiler will supply the version with `long` data type.

```
1 int x = 5;  
2 functionName<long> (x) ;
```

- We can even force the compiler to use a version that is likely to cause data loss, for example `type short`.

Example 1

```
1  template <class T>
2  void Swap(T& a, T& b);
3
4  int main() {
5
6      char a = 'A'; char b = 'Z';
7      cout<<"Characters before call: "<<a<<" and "<<b<<endl;
8      Swap<char>(a,b);
9      cout<<"Characters after call: "<<a<<" and "<<b<<endl;
10
11     double x = 1.5; double y = 3.2;
12     cout<<"Doubles before call: "<<x<<" and "<<y<<endl;
13     Swap<double>(x,y);
14     cout<<"Doubles after call: "<<x<<" and "<<y<<endl;
15
16     int i = 2; int j = 5;
17     cout<<"Integers before call: "<<i<<" and "<<j<<endl;
18     Swap<int>(i,j);
19     cout<<"Integers after call: "<<i<<" and "<<j<<endl;
20
21     return 0;
22 }
```

Characters before call:A and Z
Characters after call:Z and A
Doubles before call:1.5 and 3.2
Doubles after call:3.2 and 1.5
Integers before call:2 and 5
Integers after call:5 and 2

Example 1 (Cont.)

```
1  template <class T>
2  void Swap(T& a, T& b){
3      T temp;
4      temp = b;
5      b = a;
6      a = temp;
7  }
```

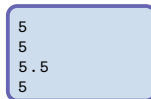
Example 2

```
1  template <class T>
2  T Square(T n){
3      T number = n*n;
4  return number;
5  }
6
7  int main() {
8
9      cout << "Square of " << 3 << " is:" << Square(3) << endl;
10     cout << "Square of " << 4.5 << " is:" << Square(4.5) << endl;
11
12     return 0;
13 }
```

Square of 3 is:9
Square of 4.5 is:20.25

Example 3

```
1  template<class T>
2  T add(T x, T y){return x+y;}
3
4  int main(){
5
6      cout<<add(2,3)<<endl; //By default an int version is created
7      //cout<<add(2,3.5)<<endl; //Error!Ambiguity:Two same types expected
8      cout<<add<int>(2,3.5)<<endl; //Forced to create an int version
9      cout<<add<double>(2,3.5)<<endl; //Forced to create a double version
10     cout<<add<long>(2,3.5)<<endl; //Forced to create a long version
11
12     return 0;
13 }
```



5
5
5.5
5

Function Templates with Multiple Types

- More than one generic type may be used in a function template. Each type must have its own parameter.

```
1 template <class T1, class T2, ... >
2 returnType functionName (parameterList) {
3 // body of function
4 }
```

Note:

- Template declarations do not occupy memory. They serve as blueprints for generating code. Memory is allocated when templates are instantiated with specific types. This is when the compiler generates the necessary code.
- Templates are a compile-time feature. The instantiation process occurs during compilation, not at runtime.
- Each instantiation of a template with different types results in a separate version of the template being generated. This can lead to code bloat if many different instantiations are used.

Example

```
1 // Function template with two different types of parameters
2 template <class T1, class T2>
3 void printPair(T1 a, T2 b) {
4     cout<<"First: "<<a<<", Second: "<<b<<endl;
5 }
6
7 int main() {
8
9     printPair(42, 3.14);           // int and double
10    printPair("Hello", 2024);      // const char* and int
11    printPair(3.14, "World");      // double and const char*
12
13    return 0;
14 }
```

```
First: 42, Second: 3.14
First: Hello, Second: 2024
First: 3.14, Second: World
```


Function Templates Overloading

- We can overload template functions by defining multiple template functions with the same name but different template parameters.
- We can also have non-template function overloads with the same name as template functions. The compiler will choose the most appropriate overload based on the arguments provided.
- When overloading templates, it's essential to ensure that the compiler can resolve which function to call unambiguously. If two overloaded functions can match the same call, it will result in a compilation error.
- In C++, non-template functions are preferred over template functions during overload resolution if they match the argument types exactly.

Example 1

```
1 // Template function to add and print two values of the same type
2 template <typename T>
3 void add(T a, T b) {
4     cout<<"Sum: "<<(a + b)<<endl;
5 }
6
7 // Template function to add and print two values of the different type
8 template <typename T1, typename T2>
9 void add(T1 a, T2 b) {
10     cout<<"Sum: "<<(a + b)<<endl;
11 }
12
13 //Non-template overload for concatenating and printing strings
14 void add(const string& a, const string& b) {
15     cout<<"Concatenation: "<<a + b<<endl;
16 }
```

Example 1 (Cont.)

```
1  int main() {  
2  
3      // Calling the function template with the same types  
4      add(3, 4);                // int and int  
5      add(3.5, 4.5);           // double and double  
6  
7      // Calling the function template with different types  
8      add(3, 4.5);              // int and double  
9      add(3.5, 4);              // double and int  
10  
11     //Calling the non-template overload for strings  
12     add(string("Hello"), string(" World")); // Two strings  
13  
14     return 0;  
15 }
```

Sum: 7
Sum: 8
Sum: 7.5
Sum: 7.5
Concatenation: Hello World

Example 2

```
1  template <class T> // Template version
2  T Square(T n){
3      cout<<"Template."<<endl;
4      return n*n;
5  }
6
7  int Square (int n) { // Non-template version
8      cout<<"Conventional."<<endl;
9      return n*n;
10 }
11
12 int main() {
13
14     cout << "Square of " << 3 << " is:" << Square(3) << endl;
15
16     return 0;
17 }
```

Conventional.
Square of 3 is:9

Function Template with Default Arguments

```
1 // Below code is strictly compiler-dependent (only most-recent compilers work)
2 // Function template with default arguments
3 template <typename T = int>
4 void printValue(T value = 0) {
5     cout<<"Value: "<<value<<endl;
6 }
7
8 int main() {
9     // Function template without providing arguments
10    printValue(); // default template and default function arguments
11
12    // Function template with an argument
13    printValue(5); // default template argument
14
15    printValue<int>(); // default function argument
16
17    // Function template with a different type and argument
18    printValue<float>(3.5); // Explicitly specifying arguments
19
20    return 0;
21 }
```

Value: 0
Value: 5
Value: 0
Value: 3.5

Class Templates

- Using a class template, we create a class that defines all the algorithms used by that class and the actual type of the data being manipulated will be specified as a parameter when objects of that class are created.
- Generic classes are useful when a class uses logic that can be generalized. For example, the same algorithms that maintain a queue of integers will also work for a queue of characters.

```
1  template <class type>
2  class className {
3  //  body of class
4  };
```

- Here, type is the type name, which will be specified when a class is instantiated. If necessary, we can define more than one generic data type using a comma-separated list.
- Once we have created a generic class, we can create an instance of that class:

```
1  className <type> obj; // 'type' is mandatory here
```

- Here, type is the type name of the data that the class will be operating upon.

Example 1

```
1  template <class T>
2  class Display{
3      T x;
4  public:
5      Display(T p) {x = p;}
6      T get(){return x;}
7  };
8
9  int main(){
10
11      Display <int> d1 (123);
12      Display <double> d2 (3.142);
13      Display <char> d3 ('A');
14      Display <string> d4 ("Hello");
15
16      cout<<"Displaying: "<<d1.get()<<endl;
17      cout<<"Displaying: "<<d2.get()<<endl;
18      cout<<"Displaying: "<<d3.get()<<endl;
19      cout<<"Displaying: "<<d4.get()<<endl;
20
21      return 0;
22 }
```

Displaying: 123
Displaying: 3.142
Displaying: A
Displaying: Hello

Example 2

```
1  template <class T>
2  class Display{
3      T x;
4  public:
5      Display(T p);
6      T get();
7  };
8  int main(){
9      Display <int> d1 (123);    Display <double> d2 (3.142);
10     Display <char> d3 ('A');   Display <string> d4 ("Hello");
11
12     cout<<"Displaying: "<<d1.get()<<endl;
13     cout<<"Displaying: "<<d2.get()<<endl;
14     cout<<"Displaying: "<<d3.get()<<endl;
15     cout<<"Displaying: "<<d4.get()<<endl;
16     return 0;
17 }
18 template <class T>
19 Display<T>::Display(T p) {x = p;}
20
21 template <class T>
22 T Display<T>::get(){return x;}
```

Displaying: 123
Displaying: 3.142
Displaying: A
Displaying: Hello

Example 3: Class Template with Multiple Types

```
1  template <class T1, class T2>
2  class Display{
3      T1 x;
4      T2 y;
5  public:
6      Display(T1 x, T2 y);
7      T1 getX();
8      T2 getY();
9  };
10
11  int main(){
12
13      Display <int,string> d1 (123,"Programming");
14      Display <string,int> d2 ("Programming",123);
15
16      cout<<" Displaying: "<<d1.getX()<<endl;
17      cout<<" Displaying: "<<d1.getY()<<endl;
18      cout<<" Displaying: "<<d2.getX()<<endl;
19      cout<<" Displaying: "<<d2.getY()<<endl;
20
21  return 0;
22 }
```

Displaying: 123
Displaying: Programming
Displaying: Programming
Displaying: 123

Example 3 (Cont.)

```
1  template <class T1, class T2>
2  Display<T1,T2>::Display(T1 p, T2 q) {x = p; y = q;}
3
4  template <class T1, class T2>
5  T1 Display<T1,T2>::getX(){return x;}
6
7  template <class T1, class T2>
8  T2 Display<T1,T2>::getY(){return y;}
```

Example 4: Arrays and Pointers in a Class Template

```
1  template <typename T, int SIZE>
2  class Array {
3      T elements[SIZE];
4
5      public:
6          // Set values in the array
7  void setValues(const T arr[SIZE]) {
8      for (int i = 0; i < SIZE; ++i) {
9          elements[i] = arr[i];
10     }
11 }
12
13     // Display values using a pointer
14 void displayValues() const {
15     const T* ptr = elements;
16     for (int i = 0; i < SIZE; ++i) {
17         cout << *(ptr + i) << " ";
18     }
19     cout << endl;
20 }
21 };
```

Integer Array: 1 2 3 4 5
String Array: Hello World C++

Example 4 (Cont.)

```
1  int main(){
2
3      Array<int, 5> intHolder;
4      int intValues[] = {1, 2, 3, 4, 5};
5      intHolder.setValues(intValues);
6
7      cout<<"Integer Array: ";
8      intHolder.displayValues();
9
10
11     Array<string, 3> stringHolder;
12     string strValues[] = {"Hello", "C++", "World"};
13     stringHolder.setValues(strValues);
14
15     cout<<"String Array: ";
16     stringHolder.displayValues();
17
18     return 0;
19 }
```

Example 5: Static Members of a Class Template

```
1 template <typename T>
2 class Counter {
3     T value;
4     static int count; //Static member
5 public:
6     Counter(T val) : value(val) {++count; }
7     ~Counter() {--count; }
8
9     void displayValue() const {
10         cout<<"Value: "<<value<<endl;
11     }
12     static int getCount() {return count; }
13 };
14
15 template <typename T>
16 int Counter<T>::count = 0;
```

```
intCounter1: Value: 1
intCounter2: Value: 2
doubleCounter1: Value: 3.5
Count for int instances: 2
Count for double instances: 1
```

Example 5 (Cont.)

```
1  int main() {
2
3      Counter<int> intCounter1(1);
4      Counter<int> intCounter2(2);
5      Counter<double> doubleCounter1(3.5);
6
7      cout<<"intCounter1: ";
8      intCounter1.displayValue();
9      cout<<"intCounter2: ";
10     intCounter2.displayValue();
11     cout<<"doubleCounter1: ";
12     doubleCounter1.displayValue();
13
14     cout<<"Count for int instances: ";
15     cout<<Counter<int>::getCount()<<endl;
16     cout<<"Count for double instances: ";
17     cout<<Counter<double>::getCount()<<endl;
18
19     return 0;
20 }
```

Example 6: Default Template Argument Values

```
1 // A Class template with default values
2 template <typename T1 = int, typename T2 = double>
3 class Pair {
4     T1 first;
5     T2 second;
6
7 public:
8     // Constructor with default values
9     Pair(T1 f = T1(), T2 s = T2()):first(f), second(s) {}
10    /* The parameter f is of type T1 and defaults to T1(), which means it is
11       initialized using the default constructor of T1. Similarly, the parameter
12       s is of type T2 and defaults to T2(), initialized using the default
13       constructor of T2. If we do not provide any arguments when creating a
14       Pair object, T1 f = T1() and T2 s = T2() will default to the default
15       constructor of T1 and T2, respectively. If T1 is int and T2 is double,
16       T1() will initialize f to 0 and T2() will initialize s to 0.0. */
17
18    void display() const {
19        cout<<"First: "<<first<<"", Second: "<<second<<endl;
20    }
21};
```

First: 0, Second: 0
First: , Second:
First: Hello, Second: World
First: 42, Second: 3.14

Example 6 (Cont.)

```
1  int main() {
2
3      // Create a Pair with default types and default values
4      Pair<>p1; //An int default value = 0, A double default value = 0.0
5      p1.display();
6
7      // Create a Pair with specified types and default values
8      Pair<string,string>p2; //A string default value is an empty string
9      p2.display();
10
11     // Create a Pair with specified types and specified values
12     Pair<string, string> p3("Hello", "World");
13     p3.display();
14
15     // Create a Pair with default types and specified values
16     Pair<> p4(42, 3.14);
17     p4.display();
18
19     return 0;
20 }
```


Example 7: Default Template Argument Values

```
1 // A Class template with default values
2 template <typename T1 = int, typename T2 = double>
3 class Pair {
4     T1 first;
5     T2 second;
6
7     public:
8         // Default arguments using parameterized constructors
9         Pair(T1 f = 10, T2 s = 20.5):first(f),second(s) {}
10
11         void display() const {
12             cout<<"First: "<<first<<" , Second: "<<second<<endl;
13         }
14 };
```

First: 10, Second: 20.5
First: Hello, Second: World
First: 42, Second: 3.14

Example 7 (Cont.)

```
1  int main() {
2
3      // Create a Pair with default types and default values
4      Pair<>p1; //Uses T1(10) and T2(20.5) for default values
5      p1.display();
6
7      // Following line wont work because default values are not compatible
8      //Pair<string,string>p2; p2.display();
9
10
11     // Create a Pair with specified types and specified values
12     Pair<string, string> p3("Hello", "World");
13     p3.display();
14
15     // Create a Pair with default types and specified values
16     Pair<> p4(42, 3.14);
17     p4.display();
18
19     return 0;
20 }
```

Example 8: Class Templates and Inheritance

```
1 // Base class template
2 template <typename T>
3 class Base {
4     protected:
5         T value;
6     public:
7         Base(T val) : value(val) {}
8         void display() const {cout<<"Base value: "<<value<<endl;}
9 };
10
11 template <class T>
12 class Derived : public Base<T> {
13     public:
14         // Constructor to initialize value using base class constructor
15         Derived(T val) : Base<T>(val) {}
16         void doubleValue() {value *= 2;}
17
18         // Overriding the display function
19         void display() const
20             {cout<<"Derived value: "<<value<<endl; }
21 };
```

Base value: 5
Derived value: 10
Derived value: 20
Derived value: 10.5
Derived value: 21

Example 8 (Cont.)

```
1  int main() {  
2  
3      // Create an instance of the base class with an integer  
4      Base<int> base(5);  
5      base.display();  
6  
7      // Create an instance of the derived class with an integer  
8      Derived<int> derived1(10);  
9      derived1.display();  
10     derived1.doubleValue();  
11     derived1.display();  
12  
13     // Create an instance of the derived class with a double  
14     Derived<double> derived2(10.5);  
15     derived2.display();  
16     derived2.doubleValue();  
17     derived2.display();  
18  
19     return 0;  
20 }
```

Example 9: Class Templates and Inheritance

```
1  template <typename T>
2  class Father {
3      protected:
4          T father;
5      public:
6          Father(const T& data) : father(data) {}
7          void display() const {
8              cout<<"Father Display: "<<father<<endl;
9          }
10 };
11
12 template <typename T>
13 class Mother {
14     protected:
15         T mother;
16     public:
17         Mother(const T& data) : mother(data) {}
18         void display() const {
19             cout<<"Mother Display: "<<mother<<endl;
20         }
21 };
```

Child Display:
Father Display: 10
Mother Display: 20

Example 9 (Cont.)

```
1  template <typename T>
2  class Child : public Father<T>, public Mother<T> {
3      public:
4          Child(const T& father, const T& mother):
5              Father<T>(father), Mother<T>(mother) {}
6
7          // Override to avoid ambiguity
8          void display() const {cout<<"Child Display: "<<endl;}
9  };
10
11  int main() {
12
13      // Create an instance of Child
14      Child<int> child(10, 20); // Pass data for Father and Mother
15
16      child.display(); // Calls Child's display
17      child.Father<int>::display(); // Calls Father's display
18      child.Mother<int>::display(); // Calls Mother's display
19
20  return 0;
21 }
```

Example 10: Class Templates and Inheritance

```
1  template <typename T>
2  class Base {
3      protected:
4          T value;
5      public:
6          Base(T val) : value(val) {}
7          void display() const {cout << "Base: " << value << endl; }
8  };
9  template <typename T>
10 class Derived1 : public virtual Base<T> {
11     public:
12         Derived1(T val) : Base<T>(val) {}
13         void display() const
14             {cout << "Derived1: "; Base<T>::display(); }
15 };
16 template <typename T>
17 class Derived2 : public virtual Base<T> {
18     public:
19         Derived2(T val) : Base<T>(val) {}
20         void display() const
21             {cout << "Derived2: "; Base<T>::display(); }
22 };
```

Diamond:
Derived1: Base: 10
Derived2: Base: 10

Example 10 (Cont.)

```
1  template <typename T>
2  class Diamond : public Derived1<T>, public Derived2<T> {
3      public:
4          Diamond(T val) : Derived1<T>(val), Derived2<T>(val),
                        Base<T>(val) {}
5          void display() const {
6              cout << "Diamond: \n";
7              Derived1<T>::display(); // display() of Derived1
8              Derived2<T>::display(); // display() of Derived2
9          }
10 };
11
12 int main() {
13
14     // Create an instance of Diamond
15     Diamond<int> diamond(10);
16     diamond.display(); // Calls display() of Diamond
17
18     return 0;
19 }
```


Example 11: Class Templates and Polymorphism

```
1  template <typename T>
2  class Shape {
3  public:
4      virtual void draw() const = 0;
5  };
6
7  template <typename T>
8  class Circle : public Shape<T> {
9  public:
10     void draw() const {cout<<"Drawing Circle"<<endl;}
11 };
12
13 template <typename T>
14 class Square : public Shape<T> {
15 public:
16     void draw() const {cout<<"Drawing Square"<<endl;}
17 };
```

Drawing Circle
Drawing Square

Example 11 (Cont.)

```
1
2 int main() {
3     Shape<int>* shape; // Pointer to the base class template
4     shape = new Circle<int>(); //Instantiate Circle and draw
5     shape->draw();
6     delete shape; // Freeing allocated memory
7
8     shape = new Square<int>(); //Instantiate Square and draw
9     shape->draw();
10    delete shape; // Freeing allocated memory
11    return 0;
12 }
```

Conclusions

In this lecture we discussed about

- Function Templates
- Class Templates

In the next lecture we will discuss about

- Exception Handling